

Infraestructura como código

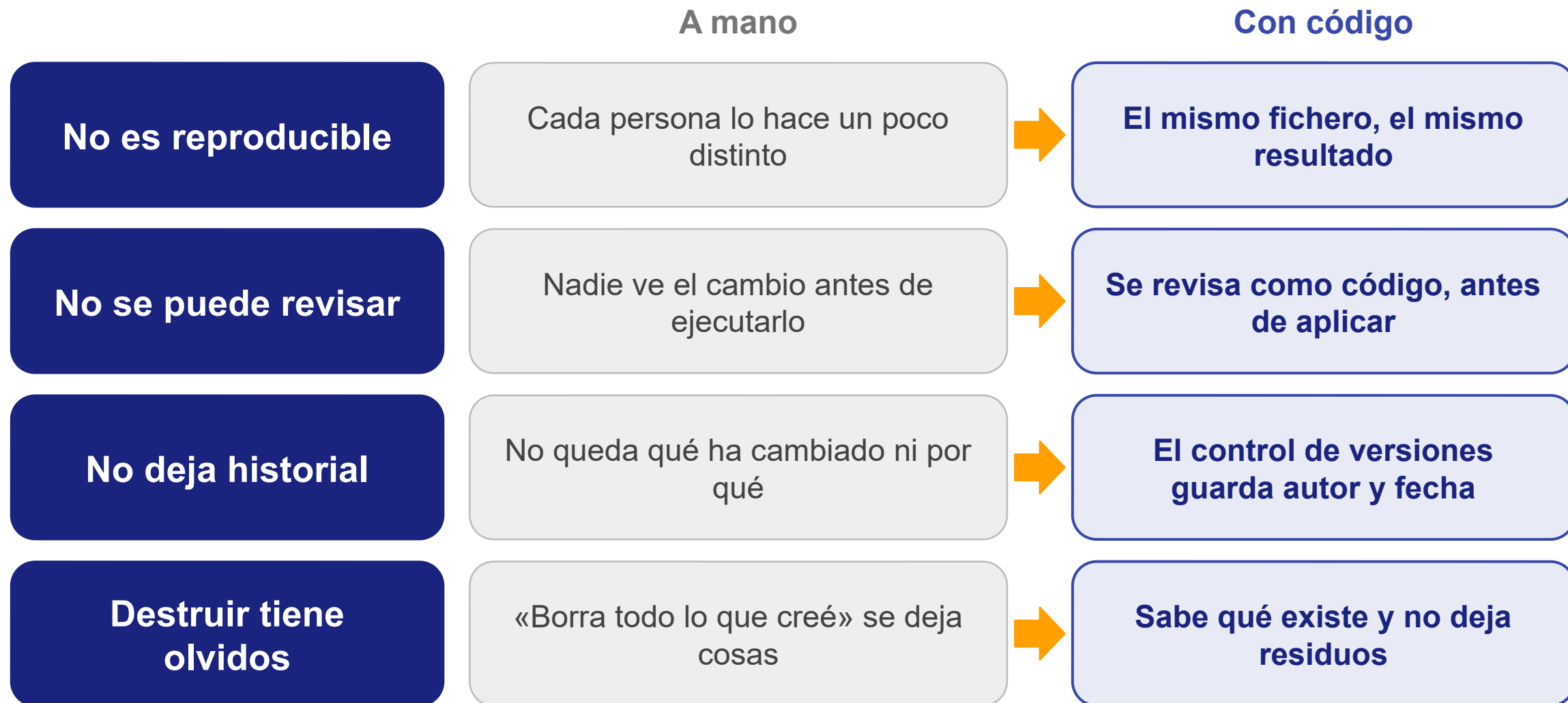
Infraestructura como código

Describir la infraestructura en ficheros de texto y dejar que una herramienta la cree, la modifique o la destruya exactamente como el texto dice.



Desde el Tema 3 has ejecutado `apply` y `destroy` sin abrir el fichero. Hoy lo lees, lo modificas y lo escribes tú.

Por qué en una empresa nadie monta producción a mano



Un olvido que ya has visto de cerca

```
terraform destroy

$ terraform destroy
aws_instance.web: Destruction complete
aws_subnet.publica: Destruction complete

aws_vpc.principal: Still destroying...
aws_vpc.principal: Still destroying...
aws_vpc.principal: Still destroying...
    ...hasta que lo borras tú
```



Actividad 3.1 y cierre de la 5.3

Los grupos de seguridad creados a mano para la base de datos, el balanceador y las instancias no están en la lista de Terraform: no los conoce.

Cuando una parte vive en código y otra a mano, el código solo responde de lo suyo.

Modelo declarativo: estado deseado, plan y aplicación

Terraform lee ficheros .tf (lenguaje HCL) y calcula qué crear, cambiar o destruir.

Imperativo: un script

Ejemplo: recrear-alb.sh de la 5.1

1. Crea la VPC



2. Crea la subred



3. Crea la pasarela



4. Crea la ruta

Si lo ejecutas dos veces, repite los pasos

Declarativo: Terraform

Ejemplo: el módulo red-base del Tema 3

Quiero una VPC con una subred pública



Compara con lo que existe y calcula la diferencia



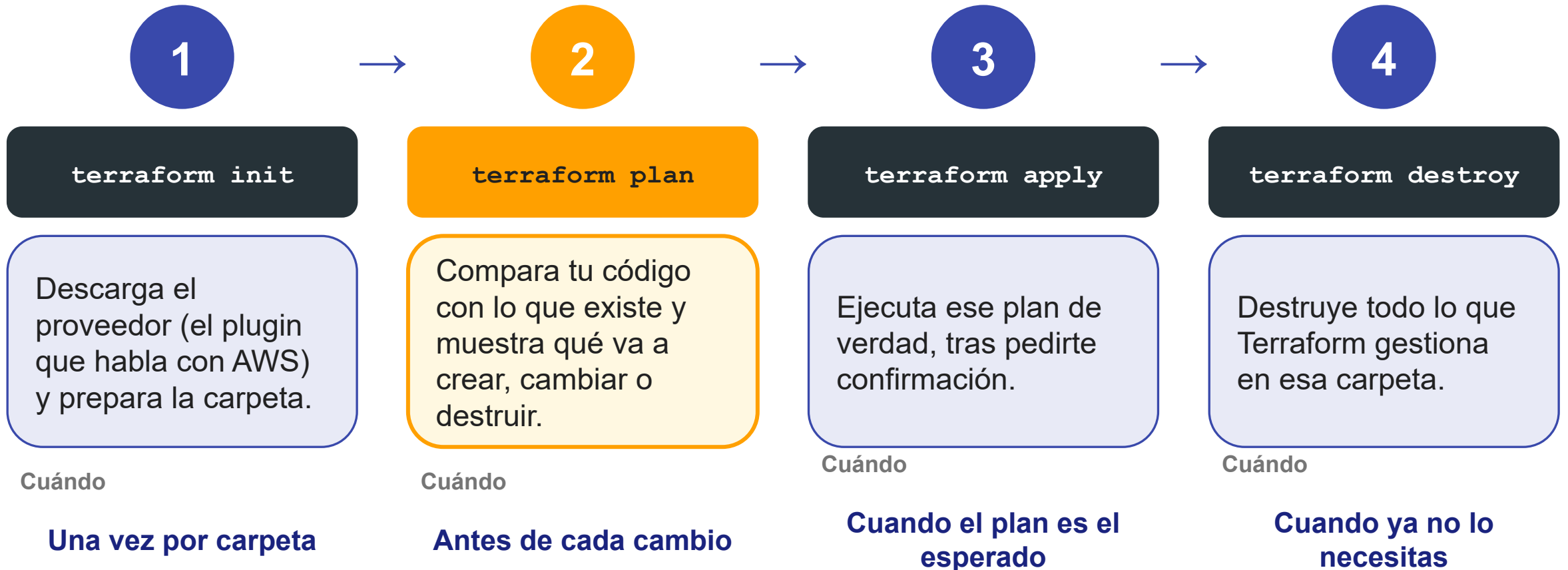
crear

cambiar

destruir

Si lo ejecutas dos veces, solo hace lo que falta

Las cuatro órdenes de Terraform



La segunda es la que marca la diferencia: terraform plan no toca nada.

Leer un plan: cuatro símbolos



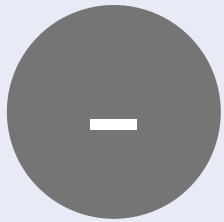
Se crea un recurso nuevo

Ejemplo: Añades una subred



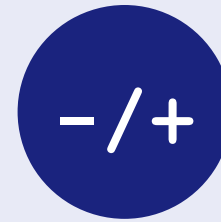
Se modifica en el sitio, sin destruirlo

Ejemplo: Cambias una etiqueta



Se destruye

Ejemplo: Quitas un recurso del código



Se destruye y se vuelve a crear

Ejemplo: Cambias el rango CIDR de una subred

El plan es como leer la lista de cambios de un contrato antes de firmarlo, en vez de firmar primero y descubrir después qué has aceptado.

Un plan real, abreviado

```
terraform plan

# aws_vpc.principal will be updated in-place
~ resource "aws_vpc" "principal" {
  ~ tags = {
    + "Curso" = "INU"
  }
}

# aws_subnet.privada must be replaced
-/+ resource "aws_subnet" "privada" {
  ~ cidr_block = "10.1.1.0/24" -> "10.2.1.0/24"
}
```

Plan: 1 to add, 1 to change, 1 to destroy.

~ La VPC

1 cambio, en el sitio

-/+ La subred

1 destrucción + 1 creación

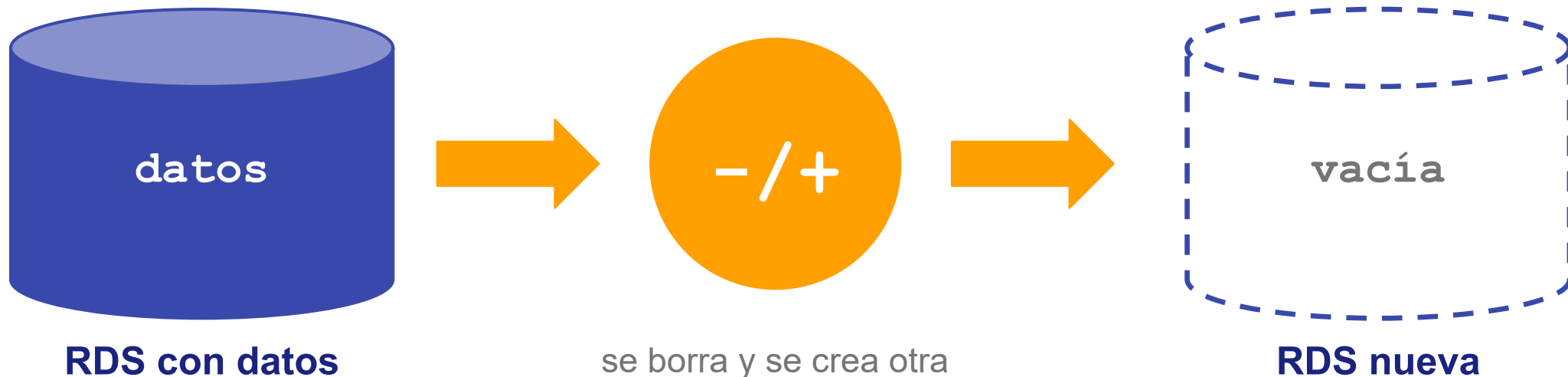
Dos recursos, tres operaciones

Lee siempre la última línea

Aplicar sin haber leído el plan

Es la forma más rápida de romper algo en producción

Un plan que dice -/+ sobre un recurso con datos significa que esos datos se pierden.



Leer el plan no es un trámite: evita que un cambio pensado como pequeño se convierta en un incidente.

Las piezas de un fichero de Terraform

provider	resource	variable	output	data
El plugin que habla con AWS, y la región donde trabaja	Una pieza concreta de infraestructura	Un valor que parametriza el código sin tocar el resto	Un valor que se muestra al terminar de aplicar	Información que lee de AWS sin crearla
En la 6.1 us-east-1	En la 6.1 Una VPC, dos subredes, una pasarela, una tabla de rutas	En la 6.1 Entorno, rango de la VPC e identificador	En la 6.1 Los ids de la VPC y de las subredes	En la 6.1 Las zonas de disponibilidad

Cada recurso de Terraform es algo que ya has creado en la consola

En la consola (la red de la 2.1)		En Terraform
VPC	→	<code>aws_vpc</code>
Subred	→	<code>aws_subnet</code>
Pasarela de internet	→	<code>aws_internet_gateway</code>
Tabla de rutas	→	<code>aws_route_table</code>
Asociar la tabla a una subred	→	<code>aws_route_table_association</code>

Así se escribe: un fragmento del módulo de la actividad

```
# Una variable: un valor que puedes cambiar sin tocar el resto del código
variable "vpc_cidr" {
  type      = string
  default   = "10.1.0.0/16"      # se usa si no indicas otro
}

# Tipo de recurso ("aws_vpc") y nombre propio dentro del código ("principal")
resource "aws_vpc" "principal" {
  cidr_block = var.vpc_cidr      # usa el valor de la variable
}

resource "aws_subnet" "publica" {
  # Referencia a la VPC de arriba: aquí nace la dependencia
  vpc_id      = aws_vpc.principal.id
  # 10.1.0.0/24: parte el /16 en trozos /24 y coge el número 0
  cidr_block  = cidrsubnet(var.vpc_cidr, 8, 0)
}
```

cidrsubnet: cómo se reparte el rango de la red

```
cidrsubnet(var.vpc_cidr, 8, 0)
```

El rango 10.1.0.0/16, partido en trozos /24

0	1	2	3	4	5	6	...
10.1.0.0/24	10.1.1.0/24	10.1.2.0/24	10.1.3.0/24	10.1.4.0/24	10.1.5.0/24	10.1.6.0/24	
subred pública	subred privada						

`var.vpc_cidr`

El rango de partida:
10.1.0.0/16

8

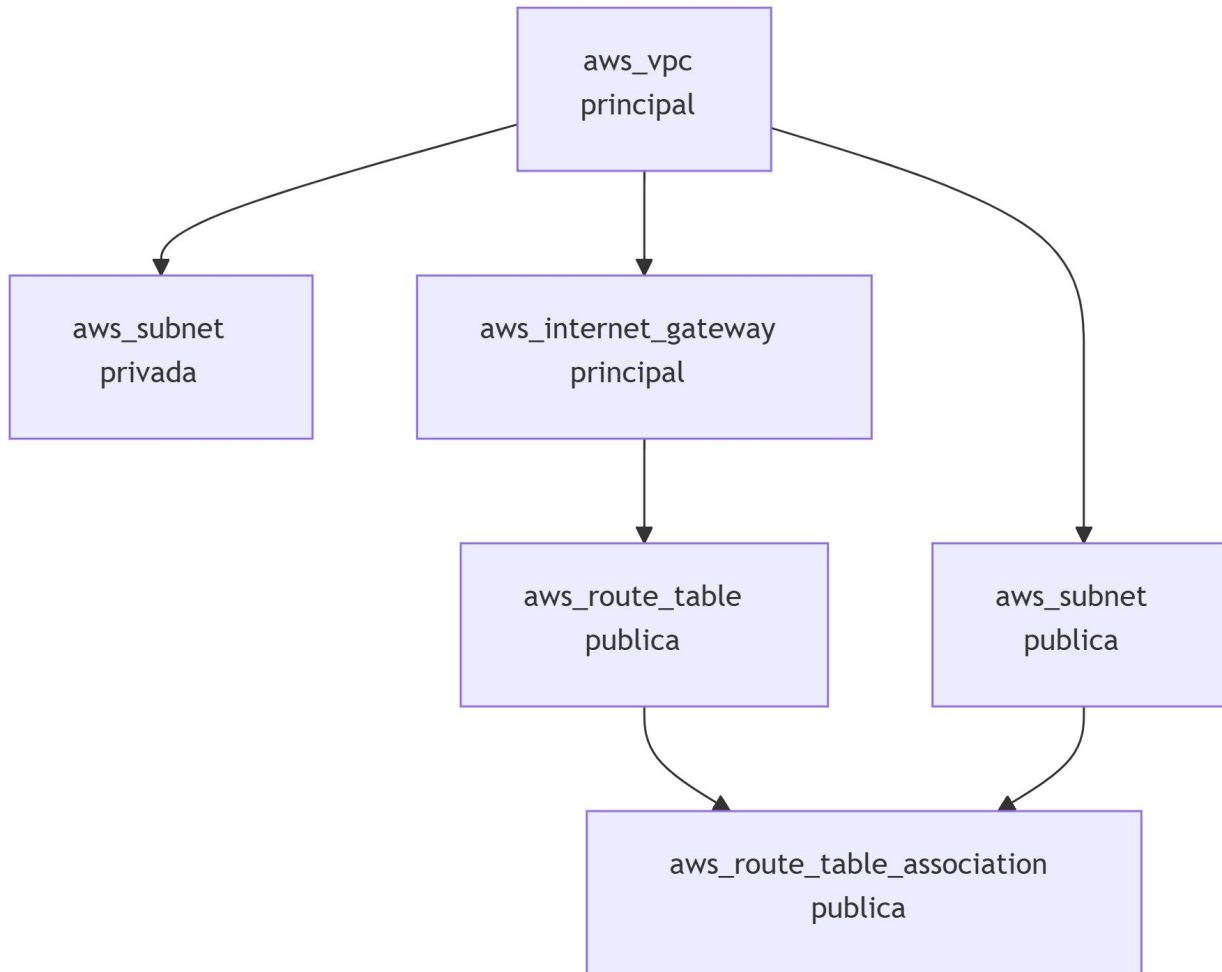
Bits que añade a la máscara:
de /16 a /24

0

Cuál de los trozos resultantes
coge

Cada subred usa un número distinto, y por eso sus rangos no se solapan.

Dependencias: el orden sale de las referencias



Referencia = dependencia

`vpc_id = aws_vpc.principal.id`: la VPC va primero

La flecha

significa «tiene que existir antes que»

Al crear

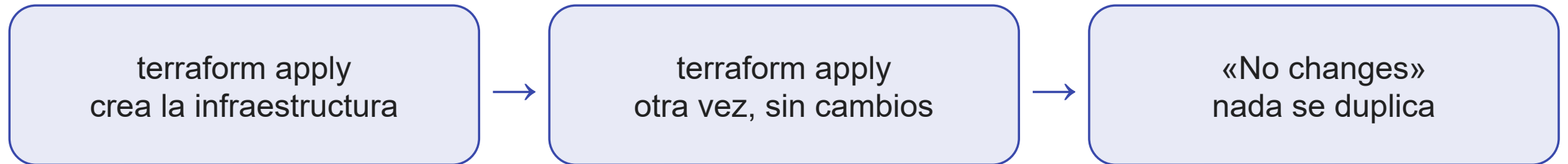
De arriba abajo, según el grafo

Al destruir: orden inverso

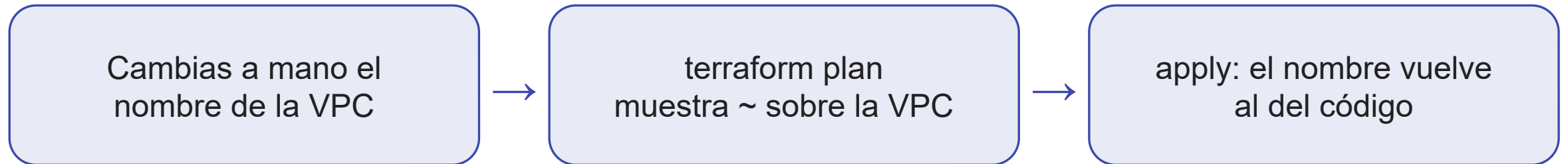
Se atasca si queda algo que Terraform no conoce

Idempotencia y deriva

Idempotencia: aplicar dos veces seguidas no cambia nada



Deriva: alguien cambia a mano algo que gestiona Terraform



Como el botón del ascensor: pulsarlo diez veces no trae diez ascensores. Y el código manda.

El fichero de estado y por qué en producción va remoto



Antes de calcular el plan, Terraform mira AWS para comprobar que lo que recuerda sigue siendo cierto, y lo compara con tu código.

Los tres mundos: qué ve el plan en cada caso

Los tres mundos de Terraform: ¿qué ve el plan en cada caso?

 no coincide con lo demás

	 Tu código lo que quieres que exista	 El estado lo que Terraform recuerda haber creado	 AWS real lo que existe de verdad	 terraform plan lo que te dice
1 Todo coincide	 VPC ✓	 VPC ✓	 VPC ✓	→ Sin cambios
2 Añades una subred al código	 VPC ✓  subred	 VPC ✓	 VPC ✓	→ + Crear la subred
3 Cambias algo a mano en la consola	 VPC nombre: dev ✓	 VPC nombre: dev ✓	 VPC nombre: prueba ✎	→ ~ Devolver el nombre a «dev»
4 Creas algo a mano en la consola	—	—	 grupo-a-mano	→  No lo ve. destroy tampoco lo borra: puede bloquear el borrado de la VPC.
terraform apply: ejecuta el plan y actualiza el estado terraform destroy: borra de AWS solo lo que está en el estado				

Cuando el estado se descuadra



Situación 1

Borras el estado con la infraestructura viva

→ Terraform olvida que existe: el siguiente apply intenta crearlo todo otra vez y lo que había queda huérfano.



Situación 2

Creas algo a mano dentro de lo que gestiona Terraform

→ No está en el estado: plan no lo ve, destroy no lo borra y puede bloquear lo que lo rodea.



Situación 3

Dos personas aplican a la vez con estados distintos

→ Se pisan el trabajo, y cada estado cree que existe algo diferente.

Mientras exista infraestructura, no edites ni borres el fichero de estado.

Estado local en la práctica, remoto en una empresa

En este módulo: local

El fichero se queda en tu carpeta de CloudShell, junto al código. Trabajas solo y en un laboratorio temporal.



En una empresa: remoto

Un almacenamiento compartido, por ejemplo un bucket S3 con bloqueo de escritura concurrente. Todo el equipo trabaja sobre el mismo estado real.



Es una simplificación consciente: no es la práctica recomendada para un equipo.

Ideas clave

1

A mano no es reproducible ni revisable; el código sí.

2

Declarativo: describes el estado deseado y Terraform calcula la diferencia.

3

El plan enseña los cambios antes de aplicar: ~ en el sitio, -/+ destruye y crea.

4

Proveedor, recursos, variables, salidas y datos; el orden sale de las referencias.

5

Aplicar dos veces no cambia nada (idempotencia); lo cambiado a mano se detecta (deriva).

6

El estado registra lo que ha creado Terraform: lo creado a mano no está, y destroy no lo toca.

Con esto ya tienes las piezas para la Actividad 6.1 — Destruir y reconstruir.